

Relatório do Projeto

Sistema de Gerenciamento de Gêneros Musicais e Artistas

Fernando Rodrigues, Orlando Luiz de Carvalho, Vinícius Albino

3 de agosto de 2026

Sumário

1	Introdução	2
2	Documentação do Código	2
2.1	Estruturas utilizadas	2
2.2	Funções de criação de lista, gêneros e artistas	2
3	Exemplos de Uso	11
3.1	Inserindo um novo gênero	11
3.2	Inserindo um artista	12
3.3	Buscando um gênero	13
3.4	Removendo um artista	14
3.5	Consulta por número mínimo de premiações	15
4	Conclusão	16

1 Introdução

O objetivo do projeto foi desenvolver um sistema capaz de armazenar e gerenciar informações sobre gêneros musicais e seus respectivos artistas utilizando listas encadeadas em linguagem C.

Após a análise do problema, percebeu-se que existia uma relação hierárquica entre os dados: um gênero pode possuir vários artistas, enquanto cada artista pertence a apenas um gênero. Dessa forma, foi adotada uma estrutura composta por duas listas duplamente encadeadas.

A primeira estrutura corresponde à lista principal, responsável pelo armazenamento dos gêneros musicais. Cada nó dessa lista contém informações do gênero, além de ponteiros para o início e o fim da lista de artistas pertencentes àquele gênero.

A segunda estrutura representa a lista de artistas, onde cada nó armazena informações como nome, cidade de origem, quantidade de integrantes, quantidade de premiações, ano de estreia, situação de atividade e principais obras.

Além das listas, o sistema realiza leitura automática de dois arquivos texto: um contendo os gêneros musicais e outro contendo os artistas. Durante essa leitura, cada artista é inserido na lista correspondente ao seu gênero.

A solução desenvolvida permite executar operações de inserção, busca, alteração, remoção e consultas específicas, mantendo a organização dos dados por meio das listas encadeadas.

2 Documentação do Código

O sistema foi dividido em módulos para facilitar sua organização.

2.1 Estruturas utilizadas

Foram implementadas três estruturas principais:

- **Lista principal:** representa a lista de gêneros.
- **Gênero:** representa cada gênero musical e contém uma lista de artistas.
- **Artista:** representa um artista ou banda pertencente a um gênero.

Essa organização caracteriza uma lista encadeada de listas.

2.2 Funções de criação de lista, gêneros e artistas

`criarListaPrincipal()`

Responsável por criar a lista principal e inicializar seus ponteiros.

```

ListaPrincipal* criarListaPrincipal(void) {
    // Inicia a lista principal (de gêneros)
    ListaPrincipal* lp = (ListaPrincipal*) malloc(sizeof(ListaPrincipal));

    if (lp == NULL) {
        mensagemDeErro("Erro de memória. Rode o programa novamente.");
    }

    lp->inicio = NULL;
    lp->fim = NULL;
    lp->qtdGeneros = 0;

    return lp;
}

```

criarGenero()

Aloca dinamicamente um novo gênero, verifica se o ID já existe e inicializa seus campos.

```

Genero *criarGenero(ListaPrincipal *lp, int id, const char nome[]) {
    // Cria um "nó gênero" dado um ID e um nome
    if (lp == NULL || nome == NULL) return NULL;

    if (buscarGenero(lp, id) != NULL) {
        printf("\nJá existe um gênero com esse ID no sistema!\n");
        return NULL;
    }

    Genero* novo = (Genero*) malloc(sizeof(Genero));
    if (novo == NULL) {
        printf("\nErro de alocação de memória!\n");
        return NULL;
    }

    novo->id = id;
    copiarTexto(novo->nome, nome);
    novo->inicioArtistas = NULL;
    novo->fimArtistas = NULL;
    novo->qtdArtistas = 0;

    return novo;
}

```

criarArtista()

Cria um novo artista, validando as informações recebidas e impedindo a duplicação de IDs.

```

Artista *criarArtista(ListaPrincipal *lista, int id, const char nome[], int qtdIntegrantes, int qtdPremiacoes,
                     const char cidadeOrigem[], int estreia, int atividade, int encerramento,
                     const char principaisObras[]) {
    // Cria um "nó artista" dadas as informações para preencher seus campos
    Artista *novo = NULL;

    if (lista == NULL || nome == NULL || cidadeOrigem == NULL || principaisObras == NULL) {
        return NULL;
    }

    if (localizarArtistaGlobal(lista, id) != NULL) {
        printf("Já existe um artista com esse ID no sistema.\n");
        return NULL;
    }

    novo = malloc(sizeof(Artista));
    if (novo == NULL) {
        printf("Falha na alocação de memória.\n");
        return NULL;
    }

    copiarTexto(novo->nome, nome);
    copiarTexto(novo->cidadeOrigem, cidadeOrigem);
    copiarTexto(novo->principaisObras, principaisObras);
    novo->ant = NULL;
    novo->prox = NULL;
    novo->id = id;
    novo->qtdIntegrantes = qtdIntegrantes;
    novo->qtdPremiacoes = qtdPremiacoes;
    novo->estreia = estreia;
    novo->atividade = atividade;
    novo->encerramento = encerramento;

    return novo;
}

```

Funções de inserção

inserirGenero()

Insere um gênero ao final da lista principal.

```

int inserirGenero(ListaPrincipal* lp, Genero *g) {
    // Insere um gênero na lista principal
    if (g == NULL) {
        printf("Gênero inválido.\n");
        return 0;
    }

    g->ant = lp->fim;
    g->prox = NULL;

    if (listaGenerosEhVazia(lp)) {
        lp->inicio = g;
    } else {
        lp->fim->prox = g;
    }

    lp->fim = g;
    lp->qtdGeneros++;

    return 1;
}

```

inserirArtistaGenero()

Insere um artista na lista pertencente ao gênero informado.

```
int inserirArtistaGenero(Artista *a, Genero *genero) {
    // Insere um artista no fim da lista de um determinado gênero
    if (a == NULL) {
        printf("Artista inválido.\n");
        return 0;
    }

    a->ant = genero->fimArtistas;

    if (listaArtistasEhVazia(genero)) {
        genero->inicioArtistas = a;
    } else {
        genero->fimArtistas->prox = a;
    }

    genero->fimArtistas = a;
    genero->qtdArtistas++;

    return 1;
}
```

Funções de busca de gênero e artista

buscarGenero()

Percorre a lista principal procurando um gênero pelo ID.

```
Genero* buscarGenero(ListaPrincipal* lp, int id) {
    // Busca e retorna o ponteiro para um nó gênero; em caso de falha, retorna NULL
    if (lp == NULL) return NULL;

    Genero* atual = lp->inicio;
    while (atual != NULL) {
        if (atual->id == id) {
            return atual;
        }
        atual = atual->prox;
    }
    return NULL;
}
```

buscaArtistaGenero()

Realiza a busca de um artista dentro de um gênero específico.

```

Artista *buscaArtistaGenero(Genero *genero, int id) {
    // Busca um artista pelo ID dentro da lista de um gênero específico
    if (genero == NULL) {
        return NULL;
    }

    Artista *aux;
    aux = genero->inicioArtistas;
    while (aux != NULL) {
        if (aux->id == id) {
            return aux;
        }
        aux = aux->prox;
    }

    return NULL;
}

```

localizarArtistaGlobal()

Percorre todos os gêneros até localizar determinado artista.

```

Artista *localizarArtistaGlobal(ListaPrincipal *lista, int id) {
    // Localiza um artista pelo ID em todos os gêneros do sistema
    Genero *genero;

    if (lista == NULL) {
        return NULL;
    }

    genero = lista->inicio;
    while (genero != NULL) {
        Artista *artista = buscaArtistaGenero(genero, id);

        if (artista != NULL) {
            return artista;
        }
        genero = genero->prox;
    }

    return NULL;
}

```

Funções de remoção

removerGenero()

Remove completamente um gênero da lista principal. Antes da remoção são desalocados todos os artistas pertencentes ao gênero, evitando vazamentos de memória.

```

int removerGenero(ListaPrincipal* lp, int id) {
    // Desconecta o nó da lista duplamente encadeada e libera sua memória e a de seus artistas.
    if (lp == NULL) return 0;

    // Localiza o nó a ser removido
    Genero* g = buscarGenero(lp, id);
    if (g == NULL) return 0; // Não encontrado

    // --- REARRANJO DOS PONTEIROS (DESCONEXÃO DO NÓ) ---

    // 1. Ajuste da ponta ANTERIOR
    if (g->ant != NULL) {
        // Se g não era o primeiro, faz o nó anterior apontar para o sucessor de g
        g->ant->prox = g->prox;
    } else {
        // Se g era o primeiro, o início da lista passa a ser o próximo de g
        lp->inicio = g->prox;
    }

    // 2. Ajuste da ponta PRÓXIMA
    if (g->prox != NULL) {
        // Se g não era o último, faz o nó seguinte apontar para o antecessor de g
        g->prox->ant = g->ant;
    } else {
        // Se g era o último, o fim da lista passa a ser o anterior de g
        lp->fim = g->ant;
    }

    // --- LIBERAÇÃO DA SUBLISTA DE ARTISTAS VINCULADA ---
    Artista* aAtual = g->inicioArtistas;
    while (aAtual != NULL) {
        Artista* proxArtista = aAtual->prox;
        free(aAtual);
        aAtual = proxArtista;
    }

    // Libera a memória do nó do gênero e reduz a contagem
    free(g);
    lp->qtdGeneros--;

    return 1; // Sucesso na remoção
}

```

removerArtistaGenero()

Remove um artista da lista do gênero ajustando corretamente os ponteiros anterior e próximo.

Funções de alteração

alterarGenero()

Permite modificar o nome de um gênero.

```

int removerArtistaGenero(Genero *genero, int id) {
    // Remove do gênero o artista indicado pelo ID e reajusta o encadeamento da lista
    if (genero == NULL) {
        return 0;
    }

    Artista *artista = buscaArtistaGenero(genero, id);
    if (artista == NULL) {
        return 0;
    }

    if (artista->ant == NULL) {
        genero->inicioArtistas = artista->prox;
    } else {
        artista->ant->prox = artista->prox;
    }

    if (artista->prox == NULL) {
        genero->fimArtistas = artista->ant;
    } else {
        artista->prox->ant = artista->ant;
    }

    free(artista);
    genero->qtdArtistas--;

    return 1;
}

```

```

int alterarGenero(ListaPrincipal* lp, int id, const char novoNome[]) {
    if (lp == NULL || novoNome == NULL) return 0;

    // Utiliza a função de busca para reaproveitar o ponteiro do nó
    Genero* g = buscarGenero(lp, id);
    if (g == NULL) return 0; // Retorna 0 (falha) se o gênero não existir

    // Atualiza com segurança a string do nome
    copiarTexto(g->nome, novoNome);

    return 1; // Retorna 1 para sinalizar sucesso
}

```

alterarArtistaGenero()

Atualiza todos os dados armazenados de um artista.


```

int alterarArtistaGenero(Genero *genero, int id, const char nome[], const char cidadeOrigem[],
                        const char principaisObras[], int qtdIntegrantes, int qtdPremiacoes, int estreia,
                        int atividade, int encerramento) {
    // Atualiza os dados do artista identificado pelo ID dentro de um gênero

    if (genero == NULL || nome == NULL || cidadeOrigem == NULL || principaisObras == NULL) {
        return 0;
    }

    Artista *artista = buscaArtistaGenero(genero, id);
    if (artista == NULL) {
        return 0;
    }

    copiarTexto(artista->nome, nome);
    copiarTexto(artista->cidadeOrigem, cidadeOrigem);
    copiarTexto(artista->principaisObras, principaisObras);
    artista->qtdIntegrantes = qtdIntegrantes;
    artista->qtdPremiacoes = qtdPremiacoes;
    artista->estreia = estreia;
    artista->atividade = atividade;
    artista->encerramento = encerramento;

    return 1;
}

```

Funções de exibição

Foram implementadas diversas funções responsáveis pela apresentação dos dados ao usuário:

- `exibirGenero()`;
- `exibirListaGeneros()` (*percorre a lista principal exibindo os gêneros*);
- `exibirArtista()`;
- `listarArtistasGenero()` (*percorre um gênero exibindo os artistas*);
- `exibirArtistasPorGenero()` (*exibe todos os artistas na forma de um relatório geral*).

Funções adicionais

Além das operações básicas, o sistema possui funcionalidades extras.

Filtrar artistas por premiações

A função percorre todos os gêneros exibindo apenas artistas cuja quantidade de premiações seja maior ou igual ao valor informado pelo usuário.

```

void filtrarArtistasPremiacoes(ListaPrincipal *lista, int minimoPremiacoes) {
    // Exibe os artistas do sistema cuja quantidade de premiações atinge o mínimo
    Genero *auxGenero;
    int qtdEncontrada = 0;

    if (lista == NULL) {
        return;
    }

    auxGenero = lista->inicio;
    while (auxGenero != NULL) {
        Artista *auxArtista = auxGenero->inicioArtistas;

        while (auxArtista != NULL) {
            if (auxArtista->qtdPremiacoes >= minimoPremiacoes) {
                printf("Gênero: %s\n", auxGenero->nome);
                exibirArtista(auxArtista);
                qtdEncontrada++;
            }
            auxArtista = auxArtista->prox;
        }

        auxGenero = auxGenero->prox;
    }

    if (qtdEncontrada == 0) {
        printf("Nenhum artista foi encontrado.\n");
    }
}

```

Encontrar o gênero com menor quantidade de artistas

Percorre toda a lista principal comparando a quantidade de artistas cadastrados em cada gênero. Ao final, exibe o gênero que possui o menor número de artistas (em caso de empate, mostra aquele que apareceu primeiro na lista principal).

```

void menorNumeroDeArtistas(ListaPrincipal *l) {
    // Retorna o gênero com o menor número de artistas; em caso de empate, retorna o que aparecer primeiro na lista
    if (listaGenerosEhVazia(l)) {
        printf("Lista de gêneros vazia.\n");
        return;
    }

    Genero *menorNumero = l->inicio;
    Genero *atual = menorNumero->prox;
    while (atual != NULL) {
        if (atual->qtdArtistas < menorNumero->qtdArtistas) {
            menorNumero = atual;
        }
        atual = atual->prox;
    }

    printf("\nO gênero com o menor número de artistas é o %s! Ele tem %d artistas cadastrados.\n", menorNumero->nome,
        menorNumero->qtdArtistas);
}

```

Manipulação de arquivos

Foram desenvolvidas funções responsáveis pela leitura dos arquivos:

- lerGenero();

- `carregarGeneros()` (*percorre todo o arquivo lendo os artistas*);
- `lerArtista()`;
- `carregarArtistas()` (*percorre todo o arquivo lendo os artistas*).

Essas funções permitem que o sistema seja iniciado já contendo todos os dados previamente cadastrados.

3 Exemplos de Uso

3.1 Inserindo um novo gênero

```
Seja bem-vindo ao seu catálogo de artistas e gêneros musicais!

O catálogo pré-definido em generos.txt foi carregado com sucesso!
O catálogo pré-definido em artistas.txt foi carregado com sucesso!

O que você deseja fazer?

1 - Adicionar um gênero/artista
2 - Buscar pelo ID de um gênero/artista
3 - Remover um gênero/artista
4 - Alterar os dados de um gênero/artista
5 - Exibir dados de gêneros/artistas
6 - Realizar uma consulta
0 - Encerrar o programa

|
```

```
1
Digite "0" e depois "enter" se você quer gênero ou "1" e depois "enter" se quer artista.

|
```

```
0
Digite um ID para o gênero: 100
Digite o nome do gênero: Blues
```

```
1 - Adicionar um gênero/artista
2 - Buscar pelo ID de um gênero/artista
3 - Remover um gênero/artista
4 - Alterar os dados de um gênero/artista
5 - Exibir dados de gêneros/artistas
6 - Realizar uma consulta
0 - Encerrar o programa

5
```

```
1 - Exibir um gênero por ID
2 - Exibir todos os gêneros
3 - Exibir um artista por ID
4 - Exibir todos os artistas de um gênero
5 - Exibir todos os artistas, por gênero

1

Digite o ID do gênero: 100
=====
Id do gênero: 100
Nome do gênero: Blues
Quantidade de artistas cadastrados do gênero: 0
=====
```

3.2 Inserindo um artista

```
1 - Adicionar um gênero/artista
2 - Buscar pelo ID de um gênero/artista
3 - Remover um gênero/artista
4 - Alterar os dados de um gênero/artista
5 - Exibir dados de gêneros/artistas
6 - Realizar uma consulta
0 - Encerrar o programa

1

Digite "0" e depois "enter" se você quer gênero ou "1" e depois "enter" se quer artista.

1
```

```

Digite um ID para o artista/banda: 100
Digite o nome do artista/banda: B.B. King
Digite quantos integrantes tem a banda (1 para artistas solo): 1
Digite quantos Grammys tem o artista/banda (quantidade de prêmios): 15
Digite a cidade de origem do artista/grupo: Itta Bena
Digite o ano de lançamento da primeira obra do artista/grupo: 1949
Digite 0 caso o grupo/artista não está mais em atividade/falecido(s). Digite 1 caso ainda estejam ativos: 0
Digite o ano de lançamento do(a) último(a) música/álbum: 2009
Digite algumas músicas/álbuns notáveis desse artista (use | como separador): The Thrill Is Gone | Live at the Regal | Lucille | Blues Is King
Digite o ID do gênero correspondente a esse artista. Digite -1 caso você queira voltar para consultar: 100

```

```

1 - Adicionar um gênero/artista
2 - Buscar pelo ID de um gênero/artista
3 - Remover um gênero/artista
4 - Alterar os dados de um gênero/artista
5 - Exibir dados de gêneros/artistas
6 - Realizar uma consulta
0 - Encerrar o programa

5

1 - Exibir um gênero por ID
2 - Exibir todos os gêneros
3 - Exibir um artista por ID
4 - Exibir todos os artistas de um gênero
5 - Exibir todos os artistas, por gênero

3

```

```

Digite o ID do artista: 100
Id do artista: 100
Nome do artista/grupo: B.B. King
Cidade de origem: Itta Bena
Quantidade de integrantes: 1
Ano de estreia: 1949
Em atividade: Não
Principais obras: The Thrill Is Gone | Live at the Regal | Lucille | Blues Is King
Ano de lançamento da música mais recente: 2009
Quantidade de prêmios obtidos: 15

```

3.3 Buscando um gênero

```

1 - Adicionar um gênero/artista
2 - Buscar pelo ID de um gênero/artista
3 - Remover um gênero/artista
4 - Alterar os dados de um gênero/artista
5 - Exibir dados de gêneros/artistas
6 - Realizar uma consulta
0 - Encerrar o programa

2

1 - Buscar pelo ID de um gênero
2 - Buscar pelo ID de um artista

1

```

```
Digite o nome do gênero do qual quer saber o ID: Rock
O ID do gênero Rock é 1.
```

3.4 Removendo um artista

```
1 - Adicionar um gênero/artista
2 - Buscar pelo ID de um gênero/artista
3 - Remover um gênero/artista
4 - Alterar os dados de um gênero/artista
5 - Exibir dados de gêneros/artistas
6 - Realizar uma consulta
0 - Encerrar o programa

2

1 - Buscar pelo ID de um gênero
2 - Buscar pelo ID de um artista

2
```

```
Digite o nome do artista do qual quer saber o ID: Gilberto Gil
O ID do artista Gilberto Gil é 71.
```

```
1 - Adicionar um gênero/artista
2 - Buscar pelo ID de um gênero/artista
3 - Remover um gênero/artista
4 - Alterar os dados de um gênero/artista
5 - Exibir dados de gêneros/artistas
6 - Realizar uma consulta
0 - Encerrar o programa

3

Digite "0" e depois "enter" se você quer gênero ou "1" e depois "enter" se quer artista.

1

Digite o ID do gênero correspondente ao artista: 47
Digite o ID do artista que quer remover: 71

Artista removido com sucesso.
```

```

1 - Adicionar um gênero/artista
2 - Buscar pelo ID de um gênero/artista
3 - Remover um gênero/artista
4 - Alterar os dados de um gênero/artista
5 - Exibir dados de gêneros/artistas
6 - Realizar uma consulta
0 - Encerrar o programa

2

1 - Buscar pelo ID de um gênero
2 - Buscar pelo ID de um artista

2

Digite o nome do artista do qual quer saber o ID: 71

Artista não encontrado. Adicione-o!

```

3.5 Consulta por número mínimo de premiações

```

1 - Adicionar um gênero/artista
2 - Buscar pelo ID de um gênero/artista
3 - Remover um gênero/artista
4 - Alterar os dados de um gênero/artista
5 - Exibir dados de gêneros/artistas
6 - Realizar uma consulta
0 - Encerrar o programa

6

1 - Filtrar artistas por um número mínimo de premiações (Grammys)
2 - Encontrar o gênero com o menor número de artistas (em caso de empate, consideramos o que vier primeiro na lista)

1

```

```

Por quantos Grammys vamos filtrar? 25

```

```
Gênero: Hip Hop/Rap
Id do artista: 75
Nome do artista/grupo: Kendrick Lamar
Cidade de origem: Compton
Quantidade de integrantes: 1
Ano de estreia: 2004
Em atividade: Sim
Principais obras: good kid, m.A.A.d city | To Pimp a Butterfly | DAMN. | GNX
Ano de lançamento da música mais recente: 2024
Quantidade de prêmios obtidos: 27

Gênero: R&B
Id do artista: 11
Nome do artista/grupo: Beyoncé
Cidade de origem: Houston
Quantidade de integrantes: 1
Ano de estreia: 1997
Em atividade: Sim
Principais obras: Crazy in Love | Halo | Renaissance | Cowboy Carter
Ano de lançamento da música mais recente: 2024
Quantidade de prêmios obtidos: 35

Gênero: R&B
Id do artista: 48
Nome do artista/grupo: Stevie Wonder
Cidade de origem: Saginaw
Quantidade de integrantes: 1
Ano de estreia: 1962
Em atividade: Sim
Principais obras: Songs in the Key of Life | Innervisions | Talking Book | Hotter than July
Ano de lançamento da música mais recente: 2005
```

4 Conclusão

O desenvolvimento deste projeto permitiu aplicar na prática diversos conceitos estudados na disciplina, principalmente listas duplamente encadeadas, alocação dinâmica de memória, modularização do código e manipulação de arquivos.

Um dos maiores desafios foi manter a consistência entre a lista principal de gêneros e as listas secundárias de artistas, principalmente durante operações de remoção, onde foi necessário atualizar corretamente os ponteiros e liberar toda a memória utilizada.

Outro desafio importante foi implementar a leitura dos arquivos texto e associar corretamente cada artista ao seu gênero correspondente.

Como resultado, foi desenvolvido um sistema organizado, modular e de fácil manutenção, capaz de realizar todas as operações básicas de gerenciamento de listas, além de consultas adicionais que enriquecem sua funcionalidade.

O projeto contribuiu significativamente para o entendimento de estruturas de dados dinâmicas e demonstrou a importância da organização modular do código para facilitar futuras expansões do sistema.